

Active Face Liveness Detection Using Mediapipe

Abduqayum Rasulmuhamedov¹, Kayathri Devi Devprasad², Gaurav Aggarwal³,
Naina Chaudhary⁴, Shivani Gupta⁵, Sujit Prasad⁶

^{1,2,3}Information Technology & Engineering, Amity University in Tashkent, Tashkent, Uzbekistan

⁴Amity University in Tashkent, Tashkent, Uzbekistan

⁵Nordic Premium Education, Tashkent, Uzbekistan

⁶Amity University, Uttar Pradesh, India

E-mail: ¹rabduqayum@mail.ru, ²dkdevi@amity.uz, ³gaggarwal@amity.uz, ⁴nchaudhary1@amity.edu,
⁵shivanigupta53@gmail.com, ⁶sprasad@amity.edu

Abstract—Active liveness detection is one of the important aspects of biometric authentication that helps us detect whether a person is alive and physically present in authentication process. In this paper, we are going to discuss all the details of active liveness detection system as well as passive liveness system, and we will make our own active liveness detection system using Mediapipe library of python. Active liveness detection system requires the users to perform specific actions such as moving their head left, right, up and down, smiling and blinking. If the user successfully performs the above actions, we will ensure they are present during the authentication process. We will use Mediapipe library to detect facial landmarks and specific patterns to find head movements and facial expressions. We will use Eye Aspect Ratio (EAR) to detect blinking action. To measure the distance between landmark points, we will use Euclidean distance formula. In the same way, we will find happiness using the Happiness Score formula with the specific landmark points.

Keywords: Active and Passive Liveness Detection, Mediapipe, Eye Aspect Ratio

I. INTRODUCTION

Every day, many fraudulent attempts are detected all over the world. But how do people know whether it is a fraudulent attempt or not? Many biometric systems use numerous ways to immediately detect fraud cases, for example: Liveness detection. Liveness detection system ensures that the biometric data being captured is from a living person rather than a spoofed or fabricated source. Therefore, it serves as a critical component in many biometric systems.

In this paper, we focus on exploring the intricacies of active liveness detection by comparing it with passive liveness detection, which analyzes intrinsic features of biometric traits. Active liveness detection requires the user to engage in specific actions or responses actively, while passive methods offer non-intrusive means of verifying liveness. We aim to discuss the principles, techniques, and challenges associated with active liveness detection by highlighting its potential applications in security, surveillance, and human-computer interaction domains listed in [1].

II. LITERATURE SURVEY

Biometric authentication of a person using Active liveness detection will prompt the users to blink their eyes, speak some phrases or to show some movements like moving our face or head left, right, up or down. One such kind of example is listed in Fig. 1. Nowadays, in most of the applications active liveness detection is introduced. One such example is one notable usecase in UzCard banking applications which asks the authenticated customer to turn his/her head to turn right/left to access the application further while including the viza card[1]. Active liveness detection varies from former passive methods in that it requires the user to perform specific actions or tasks to confirm their liveness actively[1,15,16]. These actions may include facial movements, vocalizations, or other physical gestures, which are then analyzed by the authentication system to validate liveness. Unlike passive methods as in Fig. 2., which rely on to intrinsic features of biometric traits, active liveness detection adds an interactive component to the authentication process.

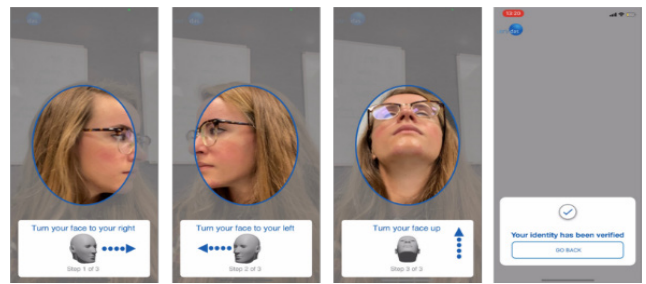


Fig. 1: Active Liveness Detection

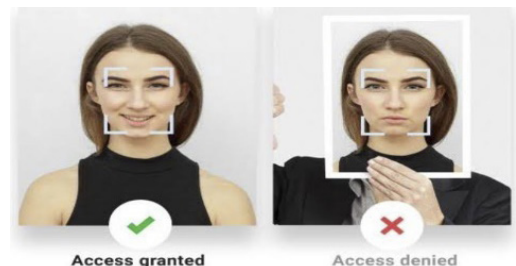


Fig. 2: Passive Liveness Detection

Active liveness detection adds an interactive element to the authentication process which will enhance security against different kind of attacks. By understanding the strengths and limitations of both approaches, practitioners can design more robust and effective liveness detection systems for various applications.

III. SYSTEM ARCHITECTURE

Our approach to make Active Liveness Detection: We used Mediapipe library to detect facial landmarks and specific patterns to find head movements and facial expressions. We used Eye Aspect Ratio (EAR) to detect blinking action. To measure the distance between landmark points, we used Euclidean distance formula. In the same way, we found happiness using the Happiness Score formula with the specific landmark points.

$$EAR (left) = left_vertical / left_horizontal \quad (1)$$

$$EAR (right) = right_vertical / right_horizontal \quad (2)$$

And if both values are less than 0.30, it indicates that user is performing blinking action. Otherwise, the user's eyes are still open.

$$Happiness\ Score = mouth_width / mouth_eye_distance$$

If Happiness Score is greater than 0.8, output will be happy. Otherwise, it indicates that the user is not smiling. This way happiness of a person can be sensed using Mediapipe and OpenCV libraries.

In the same way, math operations are used to detect whether the user is looking right or left, looking up or down. A machine learning model is also developed to detect head movements using the values which we get from OpenCV functions. We have used several state-of-the-art machine learning algorithms such as K-Nearest Neighbors (KNN), Random Forest Classifier, XGBoostClassifier. Random Forest Classifier achieved highest accuracy on the test dataset. It achieved a 99% accuracy on the test dataset, which is a promising result.

IV. METHODOLOGY

A. Face Landmarks Detection

In order to implement an active liveness detection system, we used the Mediapipe library to detect facial landmarks. Our approach involves asking users to perform specific actions, including head movements (left, right, up, down), smiling, and blinking. The idea is to detect liveness of person during authentication process.

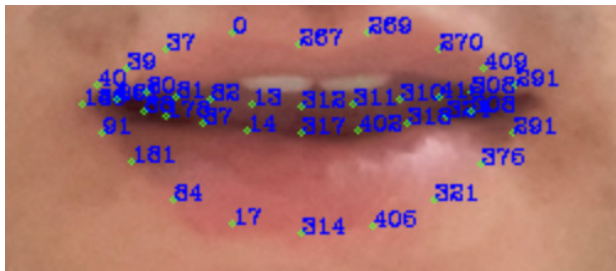


Fig. 3 : Lip Landmarks

We used Mediapipe's facial landmark detection capabilities to locate specific points on the face. This detection provides various landmark coordinates that can be used to assess facial expressions and movements.

Here is the breakdown of how we implement active liveness detection: we use landmark points from the eyes, lips, and nose.

B. Blinking Detection

Blinking detection was done by checking the Eye Aspect Ratio (EAR). We calculated the EAR for both eyes by measuring the vertical and horizontal distances between specific landmarks using Euclidean distance.

For the left eye, these landmarks are used:

Bottom (145), Top (159), Left (33), and Right (173).



Fig. 4: Left Eye Land Marked

For the *right eye*, the following landmarks are used: Bottom (374), Top (386), Left (382), and Right (249).



Fig. 5: Right Eye Land Marked

Note: Each of these landmarks has x, y and z values.

Then the Vertical distances were calculated between top and bottom landmarks for each eye. Then Horizontal distances were calculated left and right landmarks for each eye.

Finally, Eye Aspect Ratio (EAR) of both eyes was calculated using the formulas listed in Eqn. 1 & 2.

$$EAR (left) = left_vertical / left_horizontal \quad (1)$$

$$EAR (right) = right_vertical / right_horizontal \quad (2)$$

And if both values are less than 0.27, it indicates that user is performing blinking action. Otherwise, the user's eyes are still open.

C. Happiness Detection

Just like we calculated EAR to detect blinking action, we will use mathematical formulas to detect happiness. The happiness score is computed as the ratio between the mouth width and eye distance, divided by the ratio between the mouth-eye distance and the eye distance in Eqn 1. This ratio-based approach helps normalize the scores across different face sizes, providing a scale-independent measure of facial expressions.

$$\text{Happiness Score} = \frac{(\text{mouth_width} / \text{eye_distance})}{(\text{mouth_eye_distance} / \text{eye_distance})} \quad (\text{Eq. 3})$$

D. Landmarks Used

Left Eye: The coordinates are taken from landmark 33.

Right Eye: The coordinates come from landmark 263.

Mouth Left Corner: Taken from landmark 61.

Mouth Right Corner: Extracted from landmark 291.

Mouth Center: Calculated as the midpoint between the left and right mouth corners.

Mouth Width: The Euclidean distance between the left and right corners of the mouth. This gives a measure of the width of a person's smile.

Eye Distance: The distance between the left and right eyes. This is used to normalize the measurements.

Mouth-Eye Distance: The distance between the center of the mouth and the midpoint between the eyes. This measures the vertical distance between these key points. If Happiness Score is greater than 0.8, output will be happy. Otherwise, it indicates that the user is not smiling. This way happiness of a person can be sensed using Mediapipe library.

E. Head Movements Detection

In the same way, math operations can be used to detect whether the user is looking right or left, looking up or down. A machine learning model is also developed to detect head movements using the values using OpenCV.

The specific key landmarks such as those for the eyes (33 and 263), nose (1), and mouth corners (61 and 291). The coordinates of these landmarks are extracted and stored in face_2d (2D coordinates) and face_3d (3D coordinates) lists.

`camera_matrix = np.array([ [focal_length, 0, screen_height / 2], [0, focal_length, screen_width / 2], [ 0, 0, 1]])`

Diagonal Elements (focal_length): The first element represents the focal length along the x-axis; the fifth element represents the focal length along the y-axis. These determine the scale of the image.

Optical Center (screen_height/2, screen_width/2): These elements represent the optical center or principal point in the image.

Other Elements: The rest of the elements are usually zero, except for the (3,3) element, which is set to one, ensuring proper transformation.



Fig. 6: Positioning the Head with Landmarking

The `distortion_matrix` plays a crucial role in compensating for lens distortions when projecting 3D points onto a 2D image plane during pose estimation or other similar tasks. Here is how we do that.

`distortion_matrix = np.zeros((4,1), dtype=np.float)`

Then, OpenCV function called `solvePnP` which is used to estimate the pose of an object in 3D space relative to a camera, given a set of 3D points (object points) and their corresponding 2D projections (image points). It uses a camera matrix and a distortion matrix to compute the orientation and position of the object. It takes 4 parameter which are `face_3d`, `face_2d` (we defined previously), `camera_matrix` and `distortion_matrix`. This function returns 3 outputs which are `success`, `rotation_vector` and `translation_matrix`. The `rotation_matrix` indicates how much and in which direction the object is rotated. The `translation_matrix` represents the object's position relative to the camera in 3D space.

Then, `cv2.Rodrigues()` is used to take `rotation_matrix` as an input. It helps us to convert a `rotation_vector` into a `rotation_matrix`. A `rotation_vector` represents an object's rotation in 3D space, while a `rotation_matrix` provides the corresponding rotation in matrix form. It returns `rotation_matrix` and `jacobian_matrix`.

rotation_matrix: This is the output matrix that represents the same rotation as the input rotation vector.

jacobian_matrix: it contains the derivatives of the outputs with respect to the inputs. It's used in some optimization processes but is not always required for basic operations.

Finally, we will use `cv2.RQDecomp3x3()` class which takes `rotation_matrix` as an input. This class helps us to decompose a 3x3 rotation matrix into its constituent angles and other elements, providing a convenient way to interpret

the orientation represented by the rotation matrix. It returns 6 output which are angles, $mtxR$, $maxQ$, Qx , Qy , Qz .

Angles: A set of angles (usually in degrees) that represent the roll, pitch, and yaw derived from the rotation matrix.

$mtxR$, $mtxQ$: These are matrices that result from the decomposition process. They represent the different components of the original rotation matrix after applying RQ decomposition.

Qx , Qy , Qz : These are the elements of a diagonal matrix that can be extracted from the decomposition, indicating scaling factors.

We will use angles which consist of angles of each axis (x, y, z). We will have to rescale them by multiplying by 360 like this.

$x, y, z = \text{angles}[0] * 360, \text{angles}[1] * 360, \text{angles}[2] * 360$

Using these angles we will easily find whether user is looking right or left, looking up and down. For that we need to find threshold value for x and y.

Once we obtain the x and y values, we will use videos where people perform actions like looking left, right, up, down, and forward. We will gather data for each action by capturing how the x and y values change during these movements. This data will be stored in a DataFrame, including a column indicating the action associated with each data point (x, y). Finally, we will have a DataFrame containing three columns: x, y, and label.

We can then use this data to train a machine learning model. For this classification problem, following algorithms can be used: K-Nearest Neighbors (KNN), Random Forest Classifier, XGBoostClassifier, and others. We chose a Random Forest Classifier for this specific task. After training the model, we achieved a 99% accuracy on the test dataset, which is a promising result. We can now utilize this model for detecting head movements. All we need to do is pass the x and y values as input to the model. The model will then predict the corresponding action based on the provided coordinates.

V. RESULTS

Looking Right



Fig. 7: Looking Right Scenario

Looking Left



Fig. 8: Looking Left Scenario

Looking Up



Fig. 9: Looking Up Scenario

Looking Down



Fig. 10: Capturing Looking Down

Emotion: Happy



Fig. 11: Smile Action Scenario

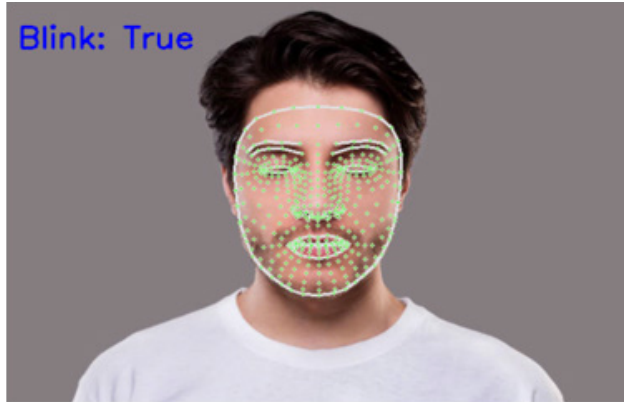


Fig. 12: Blink Action Scenario

VI. CONCLUSION

With the help of Medipipe and OpenCV libraries we have achieved amazing results. Now, this system can be used in various applications where face recognition used as this system help to detect users' liveliness.

There are also other approaches to create liveliness detection system with the help of AI. For example, we can utilize LSTM (Long Short-Term Memory) in deep learning by training the model on the dataset where people performed various actions. But it will be bit computationally expensive.

Our approach to making this system is ideal choice.

REFERENCES

- [1] Chakraborty, Saptarshi & Das, Dhrubajyoti. (2014), "An Overview of Face Liveness Detection", International Journal on Information Theory. 3. 10.5121/ijit.2014.3202.
- [2] https://developers.google.com/mediapipe/solutions/vision/face_landmarker
- [3] <https://microblink.com/glossary/active-liveness-detection/>
- [4] <https://opencv.org/>
- [5] <https://medium.com/@hotakoma/mediapipe-landmark-face-hand-pose-sequence-number-list-view-778364d6c414>
- [6] <https://www.analyticsvidhya.com/blog/2022/03/facial-landmarks-detection-using-mediapipe-library/>
- [7] J. Galbally, S. Marcel, and J. Fierrez, "Liveness detection for face recognition: A survey," *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 12, pp. 2310–2323, Dec. 2014, doi: 10.1109/TIFS.2014.2322131.
- [8] Y. Liu, A. K. Jain, and S. A. Shashidhar, "Face anti-spoofing using patch and depth-based CNN," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 2, no. 2, pp. 108–120, Apr. 2020, doi: 10.1109/TBIOM.2019.2954391.
- [9] Z. Yu, X. Li, and J. Dong, "Multi-scale 3D convolutional neural network for real-time face anti-spoofing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 5, pp. 1021–1034, Aug. 2020, doi: 10.1109/JSTSP.2020.2990727.
- [10] Y. Liu, A. K. Jain, and X. Liu, "Face anti-spoofing using convolutional neural networks and long short-term memory networks," in *2018 IEEE International Conference on Automatic Face & Gesture Recognition (FG)*, Xi'an, China, May 2018, pp. 123–130, doi: 10.1109/FG.2018.00078.
- [11] Y. Liu, J. He, X. Liu, and H. Wang, "Learning generalized deep features for face anti-spoofing via domain adaptation," in *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, Jun. 2018, pp. 1161–1168, doi: 10.1109/CVPR.2018.00612.
- [12] M. H. Mostafa, M. Elish, and A. A. Abd El-Latif, "Face liveness detection based on client identity using Siamese neural networks," *IEEE Access*, vol. 8, pp. 136763–136773, Jul. 2020, doi: 10.1109/ACCESS.2020.3013467.
- [13] Z. Yu, X. Chen, and J. Wan, "Review of face anti-spoofing techniques: Toward multimodal and generalized approaches," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 3, no. 1, pp. 68–85, Jan. 2021, doi: 10.1109/TBIOM.2021.3070684.
- [14] A. Louloudis, A. Gasteratos, and N. Nikolaidis, "Deep liveness detection for face authentication based on dynamic scene features," *Machine Vision and Applications*, vol. 32, no. 1, pp. 1–12, Jan. 2021, doi: 10.1007/s00138-020-01141-3.
- [15] Z. Yu, X. Li, and J. Dong, "DeepPixBiS: A fast and lightweight CNN-based face anti-spoofing method," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 2, no. 4, pp. 320–328, Oct. 2020, doi: 10.1109/TBIOM.2020.3020672.
- [16] G. Yin, X. Zhu, and Z. Chen, "On the effectiveness of visual and temporal features for face anti-spoofing," *Pattern Recognition Letters*, vol. 135, pp. 209–215, Aug. 2020, doi: 10.1016/j.patrec.2020.06.012.